

Université d'Ottawa
Faculté de génie

École de science informatique
et de génie électrique



University of Ottawa
Faculty of Engineering

School of Electrical Engineering
and Computer Science

Designing a UART

Project Report

CEG3155 - Digital Systems II

Prof. Rami Abielmona

FALL 2024

By:

Wajdy Elsobky - 300317535

Eknoor Goraya - 300278785

Table Of Contents

Section 1: Theoretical part-----	3
1.1: Introduction of problem -----	3
1.2. Discussion of solution -----	4
1.3 Algorithmic Solution -----	4
1.3.1 Receiver -----	4
1.3.2 Transmitter -----	8
1.3.3 Lab 3 modified -----	11
1.3.4 UART -----	13
Section 2: Design Part -----	15
2.1 Components Used -----	15
A: UART -----	15
B: UART Transmitter -----	15
C: UART Receiver -----	15
D: Baud Rate Generator -----	16
E: Auxiliary Components -----	16
2.2 Actual solution -----	16
A: Traffic Light Controller -----	16
B: UART Module Overview -----	17
C: Debug Message Generation -----	17
D: Data Flow -----	17
2.3 Discussion Of Tools -----	18
2.4 Problems Faced -----	18
Section 3: Simulations -----	19
Section 4: Discussion -----	22
Section 5: Overview -----	23
Appendix -----	24

Section 1: Theoretical part

1. Introduction to Problem

In this project, we aim to implement a synchronous UART (Universal Asynchronous Receiver-Transmitter) communication system to create a more effective debugging tool for a traffic signal controller. This builds on previous work done in the lab, with the main improvement being the addition of a transmission feature (TXD) to send information to the receiver and a reception feature (RXD) to handle incoming data from the UART.

Figure 1 shows the loopback test, an important step in validating the UART controller. This test checks the communication by redirecting transmitted signals back to the receiver without needing external hardware

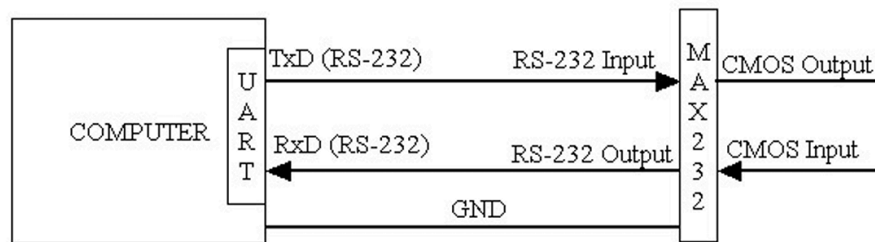


Figure 1: Loopback test of the pre-project

This process helps us better understand how the transmission and reception components work together and improves debugging for different traffic control scenarios we've seen in the lab. It has also helped us sharpen our skills in working with finite state machines (FSM) and asynchronous processes, which are key to designing digital systems.

Figure 2 illustrates the traffic light controller we're working on, highlighting the new TXD and RXD features that handle debug data transmission and reception.

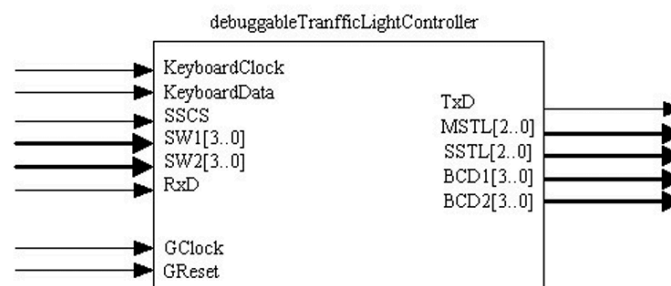


Figure 2: Traffic light controller

2. Discussion of Problem

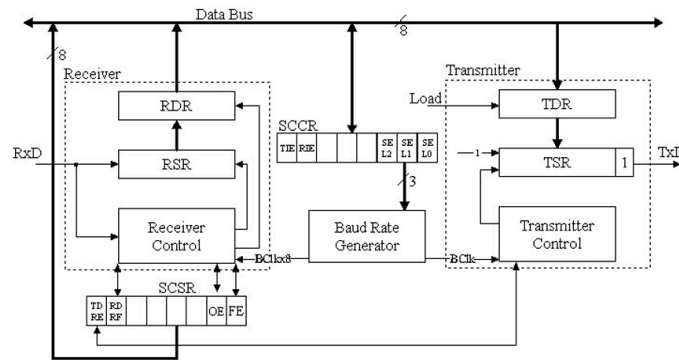
The goal of this project is to design an integrated circuit for the UART controller that can be connected to real-time sensors. The design requires creating synchronous sequential circuits, using Quartus II software and the Altera DE-2 board.

This involves carefully building all the key components of the module: the transmitter, receiver, baud generator, and control circuit. We also need to integrate this module into the traffic light control system studied in Lab 3, connecting it to the actual sensors and trigger mechanisms.

To tackle this project, we broke the design process into several phases. First, we created the transmitting and receiving units, utilizing storage and shift registers. Next, we developed a baud generator tailored to meet the technical specifications outlined in the project brief. Lastly, we worked on integrating the UART module into the traffic light control system, adding logic to enable the traffic light controllers to send status updates to the UART.

3. Algorithmic Solution

For designing the transmitter, receiver and overall UART, we utilized the FSM (Finite State Machine) approach to outline the process.

**Figure 3: UART Components**

3.1 RECEIVER

P.S	N.S								OUTPUT										
	RXD = 0	RXD = 1	CLKC TRout = 0	CLKC TRout = 1	SCTR out = 0	SCTout t = 1	RDRF = 0	RDRF = 1	CLKC TRload	CLKC TRinc	CLKC TRset	SCT R load	RFloadSR Shift R	RSRI oad	SCR Rinc	RDR Flood	RDRload	FEload	Eload
E	E	E	F	E	E	E	E	E	1	1	0	0	0	0	0	0	0	0	0
F	H	G	F	F	F	F	F	F	0	0	0	0	0	0	0	0	0	0	0
G	I	I	I	I	I	I	I	I	0	0	0	0	0	0	0	1	1	0	0
H	K	K	K	K	K	K	K	K	0	0	0	0	0	0	0	0	0	1	0
I	J	I	I	I	I	I	I	I	0	0	0	0	0	0	0	0	0	0	0
J	K	K	K	K	K	K	K	K	0	0	0	0	0	0	0	0	0	0	1
K	K	K	K	K	K	K	K	K	0	0	0	0	0	0	0	0	0	0	0

Table 2: Transmitter Transition Table

P.S	N.S								Output										
	RXD=0	RXD=1	CLKC TRout=0	CLKC TRout=1	SCTRout=0	SCTRout=1	RDRF=0	RDRF=1	CLKC TRload	CLKC TRinc	CLKC TRset	SCT R load	RFloadSR Shift R	RSRI oad	SCR Rinc	RDR Flood	RDRload	FEload	Eload
0000	0001	0000	0000	0000	0000	0000	0000	0000	0	0	0	0							0
0001	0001	0001	0010	0001	0001	0001	0001	0001	0	0	0	0							1
0010	0010	0010	0010	0010	0010	0010	0010	0010	0	0	0	0							2
0011	0011	0011	0010	0011	0011	0011	0011	0011	0	0	0	0							3
0100	0100	0100	0101	0100	0100	0100	0100	0100	0	0	0	0							4
0101	0101	0101	0101	0101	0101	0101	0101	0101	0	0	0	0							5
0110	0110	0110	0110	0110	0110	0110	0110	0110	0	0	0	0							6
0111	0111	0111	0110	0111	0111	0111	0111	0111	0	0	0	0							7
1000	1001	1000	1000	1000	1000	1000	1000	1000	0	0	0	0							8
1001	1010	1010	1010	1010	1010	1010	1010	1010	0	0	0	0							9
1010	1010	1010	1010	1010	1010	1010	1010	1010	0	0	0	0							10

From Table 2, the following simplified design equations were extracted.

$$\text{CLKCTRLsel} = y3' y2' y0'$$

$$\text{CLKCTRLload} = y3' y2 + y3' y1' y0'$$

$$\text{CLKCTRLinc} = y_3' y_2 y_0' + y_3' y_2 y_1 y_0$$

$$\text{SCTRLload} = y_3' y_2' y_1 y_0'$$

$$\text{RSRshiftR} = y_3' y_2 y_1 y_0$$

$$\text{RSRload} = y_3' y_2 y_1 y_0'$$

$$\text{SCTRLinc} = y_3' y_2' y_1 y_0$$

$$\text{RDRFload} = y_2 y_1 y_0'$$

$$\text{RDRload} = y_2 y_1 y_0'$$

$$\text{FEload} = y_3' y_2 y_1 y_0$$

$$\text{CEload} = y_3 y_0$$

$$Y_3 = (y_3' \oplus y_2) y_1 + y_3 y_2 y_0' (y_1 + \text{RDRF} \cdot y_1)$$

$$Y_2 = y_2' y_2 y_1' + y_3' y_2' y_1' y_0$$

$$Y_1 = y_2 (y_1 y_0' + y_3 y_1 y_0') (\text{CLKCTR}_{\text{out}}) + y_3' y_0 (y_1 + y_2 y_1')$$

$$Y_0 = y_3' y_2' y_0' (y_3 + \text{RXD}) + y_2' y_0 (y_3' \oplus y_3) + y_3 y_0' (\text{SCTRLload} + \text{CLKCTRL}_{\text{out}})$$

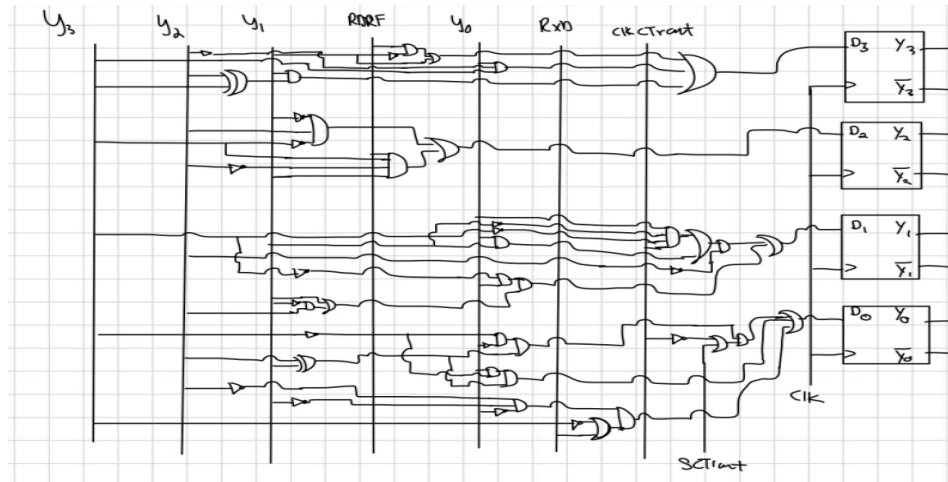


Figure 5: Receiver DFF circuit

3.2 TRANSMITTER

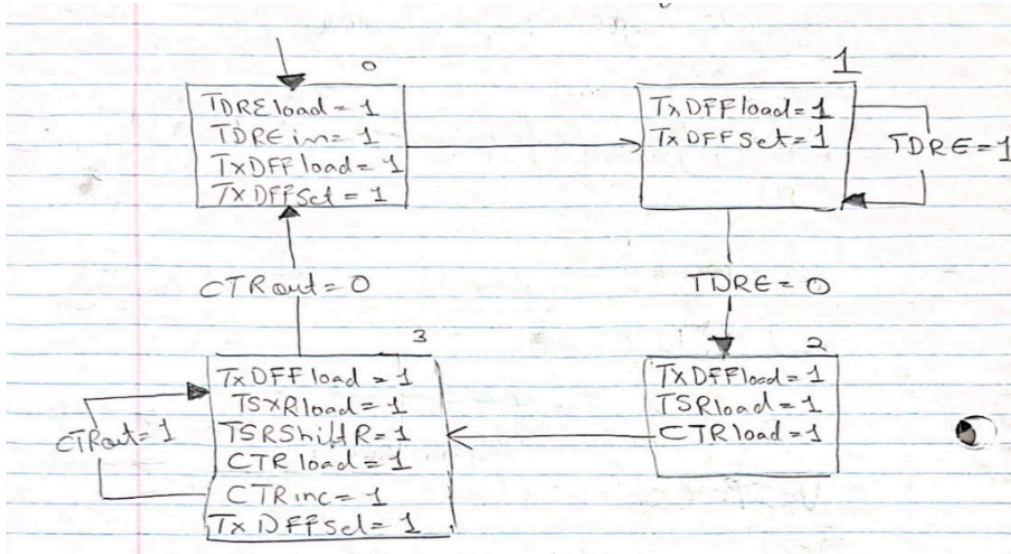


Figure 6: UART Transmitter State Diagram

The UART transmitter operates using the Transmit Data Register (TDR), Transmit Shift Register (TSR), and control logic, with its functionality defined by a four-state finite state machine (FSM). From figure 6, we can see the process begins in State 0, where the TDR is initialized, and the TDRE (Transmit Data Register Empty) status bit is set to 1 in the Serial Control Status Register (SCSR), signaling that the TDR is ready to accept data. When the microcontroller is prepared to send data, it first waits for TDRE = 1, then loads a byte into the TDR. At this point, TDRE resets to 0, indicating the register is in use. The FSM transitions to State 1, where the controller monitors the TDRE flag. Once data is confirmed in the TDR, the FSM moves to State 2, transferring the byte from the TDR to the TSR. Simultaneously, the system loads the counter to begin tracking the transmission process. In State 3, the UART outputs a start bit for one bit period, signaling the start of the transmission. The TSR then shifts its contents right, transmitting the eight data bits one by one, followed by a stop bit. A counter tracks the number of shifts, and once all bits are transmitted, the FSM resets to State 0, setting TDRE = 1 to signal readiness for the next byte.

Table 3: Transmitter State Table

Present state	Next State
---------------	------------

	TDRE = 0	TDRE = 1	CTROUT = 0	CTROUT = 1
A	A	B	A	A
B	C	B	B	B
C	C	D	C	C
D	D	D	A	D
Output				

TDREload	TDREin	TxDFFload	TxDFFset	TSRload	CTRload	TSRshiftR	CTRinc	TxDFFsel
1	1	1	1	0	0	0	0	0
0	0	1	1	0	0	0	0	0
1	1	1	0	1	1	0	0	0
0	0	1	0	1	1	1	1	1

Table 4: Transmitter Transition Table

Present State	Next State			
	Y1Y0		Y1Y0	
y1y0	TDRE = 0	TDRE = 1	CTROUT = 0	CTROUT = 1
00	00	01	00	00
01	10	01	01	01
10	10	11	10	10
11	11	11	00	11
Output				

TDREload	TDREin	TxDFFload	TxDFFset	TSRload	CTRload	TSRshiftR	CTRinc	TxDFFsel
----------	--------	-----------	----------	---------	---------	-----------	--------	----------

1	1	1	1	0	0	0	0	0
0	0	1	1	0	0	0	0	0
1	1	1	0	1	1	0	0	0
0	0	1	0	1	1	1	1	1

From the above state table, the following simplified design equations were extracted.

$$\text{TDREload} = y_0'$$

$$\text{TDREin} = y_0'$$

$$\text{TxDFFload} = 1$$

$$\text{TxDFFset} = y_1'$$

$$\text{TSRload} = y_1$$

$$\text{CTRload} = y_1$$

$$\text{TSRshiftR} = y_1 y_0$$

$$\text{CTRinc} = y_1 y_0$$

$$\text{TxDFFsel} = y_1 y_0$$

$$Y_1 = (y_1 \oplus y_0) \text{TDRE}' + y_1 y_0 + y_1 \text{CTRout}$$

$$Y_0 = y_1 \text{TDRE} + y_0 + y_1 y_0 \text{CTRout}$$

From the above design equations, the following DFF circuit was realized.

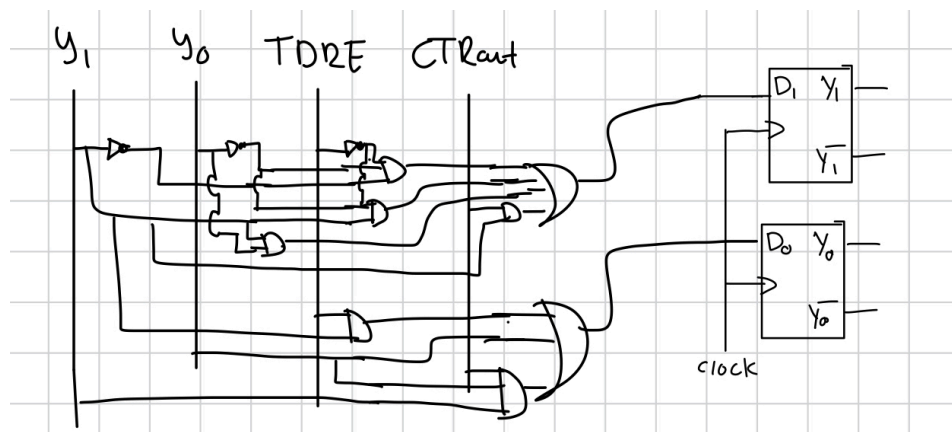


Figure 7 : Transmitter DFF Circuit

y ₁ y ₀	C0 =0	C0 =1	MT = 0	MT = 1	SSC = 0	SSC = 1	SST = 0	SST = 1	MSG	MSY	MSR	SSG	SSY	SSR
00	00	01	00	00	00	00	11	00	1	0	0	0	0	1
01	00	01	01	10	01	01	11	00	0	1	0	0	0	1
10	00	01	01	10	10	11	11	00	0	0	1	1	0	0
11	00	01	01	10	10	11	11	00	0	0	1	0	1	0

Note for Table 5 & 6, when a timer is set to 1, its expired.

From the above state table, the following simplified design equations were extracted.

$$Y_1 = y_1 + y_0 MT$$

$$Y_0 = y_1' y_0 (MSC \cdot SSC) + y_1 + y_0$$

$$MSG = y_1' y_0'$$

$$MSY = y_1 y_0$$

$$MSR = y_0' y_0 + g_1 y_0 = y_1$$

$$SSG = y_1 y_0$$

$$SSY = y_1 y_0'$$

$$SSR = y_1' y_0' + y_1 y_0 = y_1$$

$$SI = MSG \& MSY \& MSR \& SSG \& SSY \& SSR$$

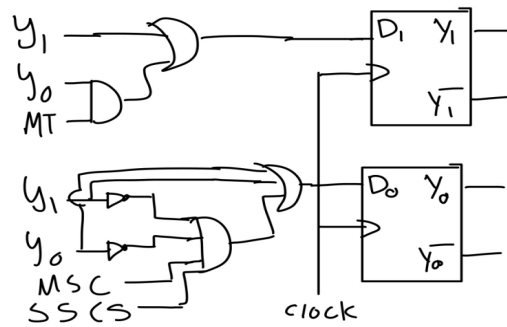


Figure 9: DFF Circuit Derived From Table 6

3.4 UART

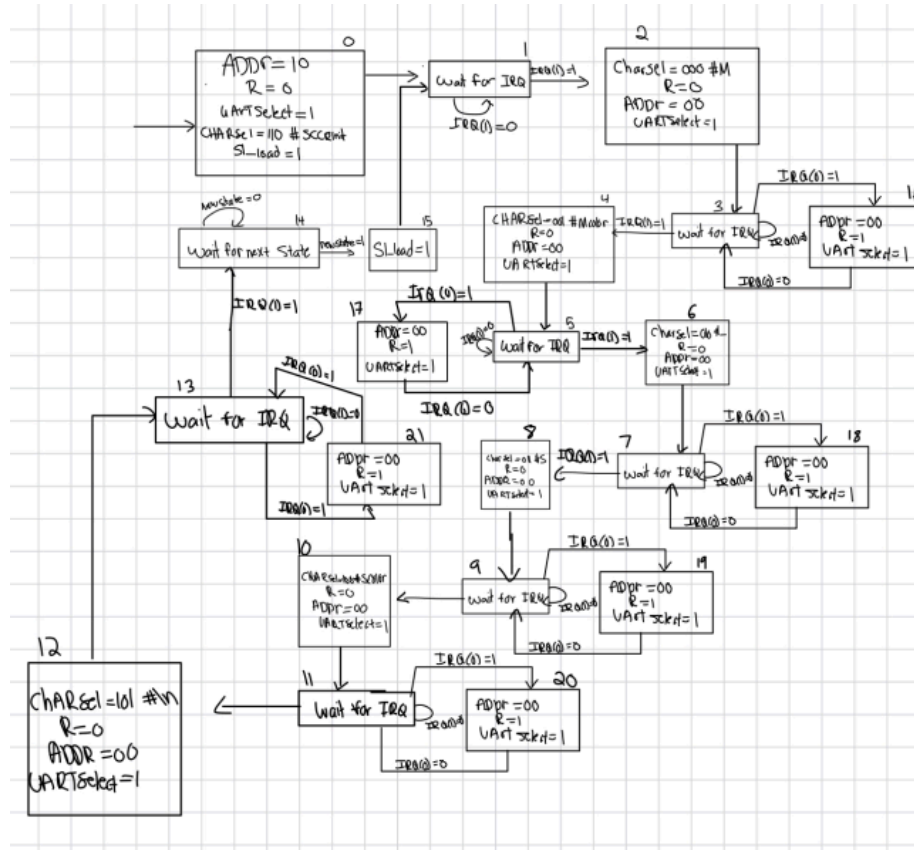


Figure 10: UART State Diagram

Due to the large number of states, it wasn't realistic to create a state/transition table. Instead, we assigned each state in the state diagram a value as you would in a transition table as well as a little $y_{stateNum}$. We then get the following design equations:

$$\text{newStateCheck} = \text{SI1out}' \cdot \text{STATE_INFORMATION}(1) + \text{SI0out}' \cdot \text{STATE_INFORMATION}(0)$$

$$\text{resetcondition} = Y_{21}' \cdot y_{20}' \cdot y_{19}' \cdot y_{18}' \cdot y_{17}' \cdot y_{16}' \cdot y_{15}' \cdot y_{14}' \cdot y_{13}' \cdot y_{12}' \cdot y_{11}' \cdot y_{10}' \cdot y_9' \cdot y_8' \cdot y_7' \cdot y_6' \cdot y_5' \cdot y_4' \cdot y_3' \cdot y_2' \cdot y_1' \cdot y_0'$$

$$Y_{21} = y_{13} \text{IRQ}(0)$$

$$Y_{20} = y_{11} \text{IRQ}(0)$$

$$Y_{19} = y_9 \text{IRQ}(0)$$

$$Y_{18} = y_7 \text{IRQ}(0)$$

$$Y_{17} = y_5 \text{IRQ}(0)$$

$$Y_{16} = y_3 \text{IRQ}(0)$$

$$Y_{15} = y_{14} \text{newStateCheck}$$

$$Y_{14} = y_{13} \text{IRQ}(1) \text{IRQ}(0)' + y_{14} \text{newStateCheck}'$$

$$Y_{13} = y_{12} + y_{13} \text{IRQ}(1)' + y_{21} \text{IRQ}(0)'$$

$$\begin{aligned}
 Y_{12} &= y_{11} \text{IRQ}(1) \text{IRQ}(0)' \\
 Y_{11} &= y_{10} + y_{11} \text{IRQ}(1)' + y_{20} \text{IRQ}(0)' \\
 Y_{10} &= y_9 \text{IRQ}(1) \text{IRQ}(0)' \\
 Y_9 &= y_8 + y_9 \text{IRQ}(1)' + y_{19} \text{IRQ}(0)' \\
 Y_8 &= y_7 \text{IRQ}(1) \text{IRQ}(0)' \\
 Y_7 &= y_6 + y_7 \text{IRQ}(1)' + y_{18} \text{IRQ}(0)' \\
 Y_6 &= y_5 \text{IRQ}(1) \text{IRQ}(0)' \\
 Y_5 &= y_4 + y_5 \text{IRQ}(1)' + y_{17} \text{IRQ}(0)' \\
 Y_4 &= y_3 \text{IRQ}(1) \text{IRQ}(0)' \\
 Y_3 &= y_2 + y_3 \text{IRQ}(1)' + y_{16} \text{IRQ}(0)' \\
 Y_2 &= y_1 \text{IRQ}(1) \\
 Y_1 &= y_0 + y_{15} + y_1 \text{IRQ}(1)' \\
 Y_0 &= \text{resetcondition}
 \end{aligned}$$

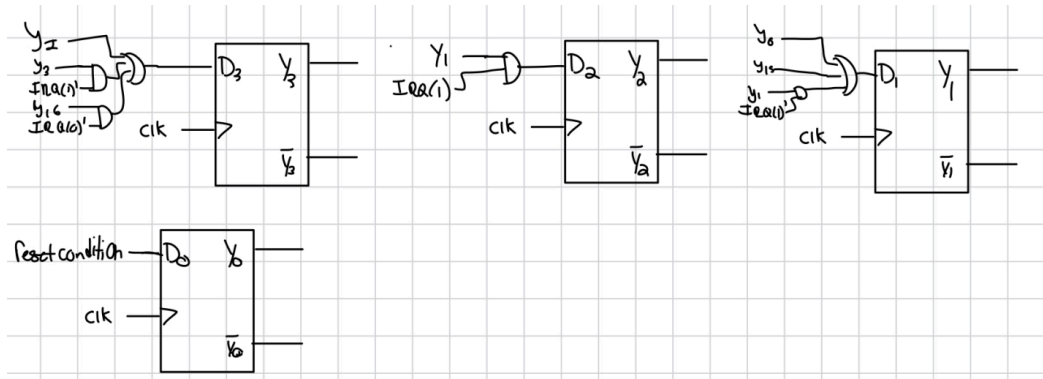


Figure 11: UART DFF circuit (0-3)

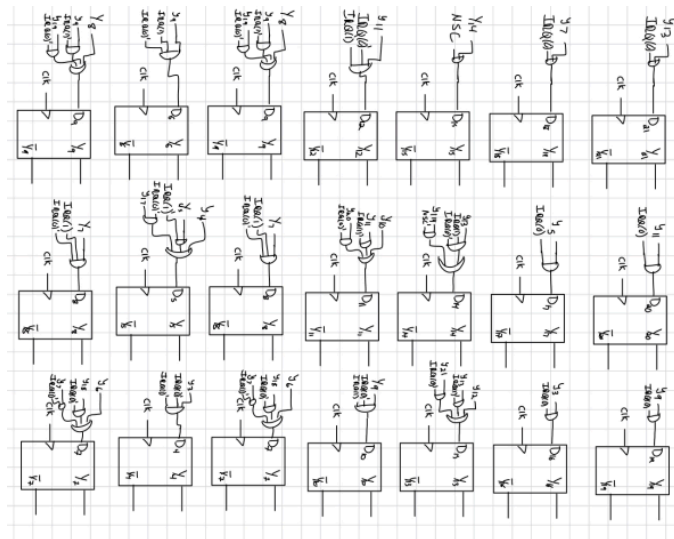


Figure 12: UART DFF circuit (4-21)

Section 2: Design Part

1. Components Used

a. UART

The UART module facilitates communication between the traffic light system and external devices, handling both data transmission (TxD) and reception (RxD). It integrates several subcomponents:

- **UART Transmitter:** Converts parallel data into a serial stream, ensuring proper framing with start, stop, and parity bits.
- **UART Receiver:** Converts serial data into parallel format and detects framing or overrun errors.
- **Baud Rate Generator:** Synchronizes the transmitter and receiver by generating clock signals at the desired baud rate.
- **Interrupt Generator:** Generates interrupts for the transmitter and receiver based on their states.
- **Address Decoder:** Decodes input addresses for efficient data routing and register control.

b. UART Transmitter

The transmitter module prepares and sends data serially via the TxD pin. Its components include:

- **Parallel-in-Serial-Out (PISO):** Converts parallel input data into a serial stream.
- **Parity Generator:** Appends a parity bit to detect transmission errors.
- **Counters and Comparators:** Track transmitted bits and indicate when a frame is complete. These elements ensure efficient and reliable data transmission.

c. UART Receiver

The receiver module processes incoming serial data from the RxD pin, converting it into parallel format for internal use. Key features include:

- **Serial-in-Parallel-Out (SIPO):** Deserializes incoming data.
- **Start Bit Detector:** Detects the beginning of a new frame.
- **Error Handling:** Ensures the integrity of received data by checking framing bits.
- **Counters and Comparators:** Track the bit position during data reception.

d. Baud Rate Generator

The Baud Rate Generator is responsible for generating the necessary clock signals for the UART transmitter and receiver, ensuring data is transmitted and received at the correct baud rate. The baud rate generator is programmable using three control bits SEL[2:0] from the SCCR register, allowing the selection of one of eight baud rates. These baud rates are defined by the system clock (25.175 MHz) and the desired baud rates, which are as follows:

- Baud Rates: 300, 600, 1200, 2400, 4800, 9600, 19200, and 38400.

e. Auxiliary Components

- **Debounce (provided by the professor):** Filter noisy input signals from switches to ensure smooth state transitions and system reliability.
- **Clock Divider (provided by the professor):** Reduces the system clock frequency to generate slower clocks.
- **Flip-Flops:** Store and propagate data within sequential logic circuits.
- **Multiplexers:** Dynamically route signals based on selection inputs.
- **Counters and Comparators:** Perform timing and conditional logic throughout the design.

2. Actual Solution

a. Traffic Light Controller Integration

The traffic light controller, initially designed in lab 3, has been extended to output state information for UART interfacing. Each of the four main states corresponds to specific traffic light configurations and debug messages:

- **State 1:** "Mg_Sr" (Main Green, Side Red)
- **State 2:** "My_Sr" (Main Yellow, Side Red)
- **State 3:** "Mr_Sg" (Main Red, Side Green)
- **State 4:** "Mr_Sy" (Main Red, Side Yellow)

This information is encoded using 6-bit state identifiers (i.e SI which is the concatenation of MSTL & SSTL), which are processed and fed as input to the UART.

b. UART Module Overview

The UART module comprises key components like the transmitter, receiver, parity generator, baud rate generator, and address decoder. These components are structurally instantiated and connected within the design:

- **Transmitter and Receiver:** The transmitter converts parallel data into serial data for debugging messages, while the receiver allows data reception via the RxD port.
- **Parity Generator:** This component uses odd parity to account for bit errors.
- **Baud Rate Generator:** This component ensures synchronization by generating the appropriate clock frequencies for UART communication.
- **Address Decoder:** Manages the control and status registers of the UART for read/write operations.

c. Debug Message Generation

The debug message is generated as a sequence of ASCII-encoded characters:

- Each message consists of six 7-bit characters (e.g., "Mg_Sr" followed by a carriage return).
- Words are pre-defined as 42-bit signals, concatenating six 7-bit ASCII characters.

Table 7: ASCII character 7 bit encoding

M	g	_	S	r	y	CR
1001101	1100111	1011111	1010011	1110010	1111001	0001101

d. Data Flow

The state information from the traffic light controller is fed into the UART component, which determines the message to send. This flow includes:

1. Using logic from lab 3, we accurately determine the traffic light state.
2. Output from step 1 is then passed as input into the UART.
3. From step 2 we can then output the correct debug message serially.

By utilizing the components mentioned above, we were able to integrate our traffic light controller from lab 3 with a UART system, together creating a debuggable traffic light controller (figure 13) that outputs debug messages unique to each state.

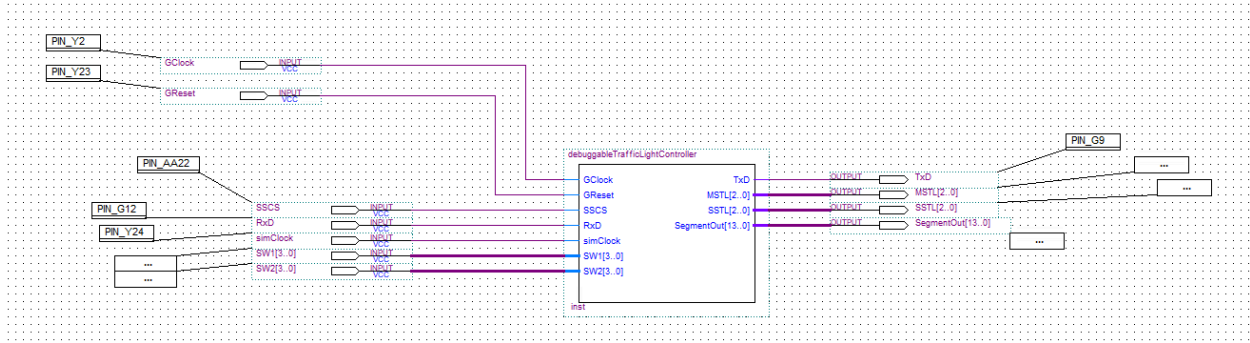


Figure 13: Top Level BDF

3. Discussion of tools

We used Quartus II CAD software for VHDL design and simulation, leveraging its advanced features to ensure circuit functionality before hardware implementation. Despite initial challenges with the Altera board, we successfully integrated hardware and software, underscoring the importance of compatibility between the two.

A key aspect of the project was implementing a loopback test using the MAX232 chip to convert CMOS signals to RS-232 levels. Following the datasheet (Figure 5, page 16), we built the circuit on a breadboard and verified voltage levels directly, ensuring proper communication between the FPGA and serial port. This hands-on testing, combined with CAD tools, was vital for ensuring signal integrity and system performance.

4. Problems Faced

The implementation of the UART-based debuggable traffic light controller faced several challenges, particularly during the transition from simulation to real hardware. While simulations ran smoothly, the integration on the Altera board revealed several issues, some of which were resolved while others required further investigation. Below are some of the problems we encountered.

1. Counter for MSC and SSC Failing on Hardware

One of the primary challenges was that the counters responsible for managing the Main Street and Side Street traffic light transitions (MSC and SSC) stopped functioning after integrating the UART system. We later discovered the issue was related to our mux selector for the baud rate generator which was selling the wrong baud rate.

2. Incorrect Data Transmission

We encountered issues earlier in our implementation which took up a significant amount of time to solve. Although the character "M" was consistently transmitted correctly, subsequent characters in the debug message were garbled or incorrect. This issue was traced to logic errors in the byte selection and transmission process.

Section 3: Simulations

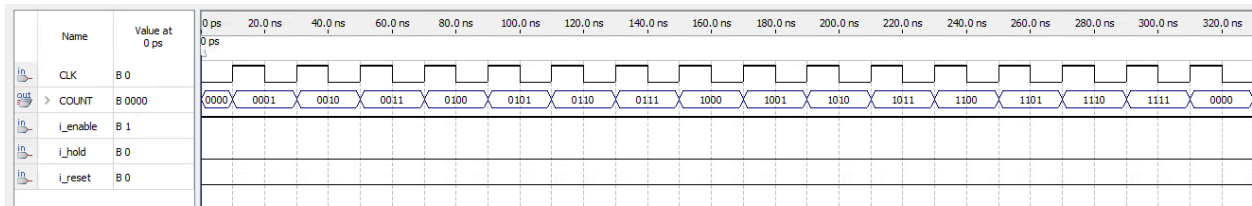


Figure 15: 4 Bit Counter

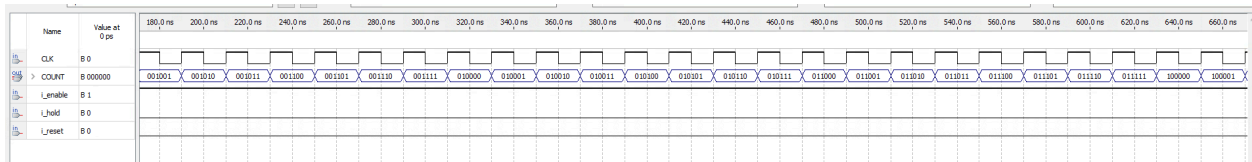


Figure 16: 6 Bit Counter

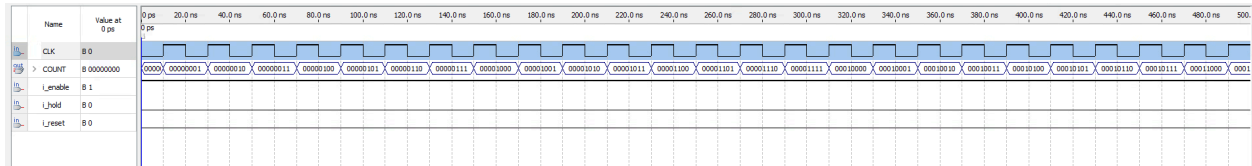


Figure 17: 8 Bit Counter

Figures 15-17 showcase the simulation of 4,6,8 bit up-counters respectively. These counters played crucial roles in our system as discussed in section 2.1.

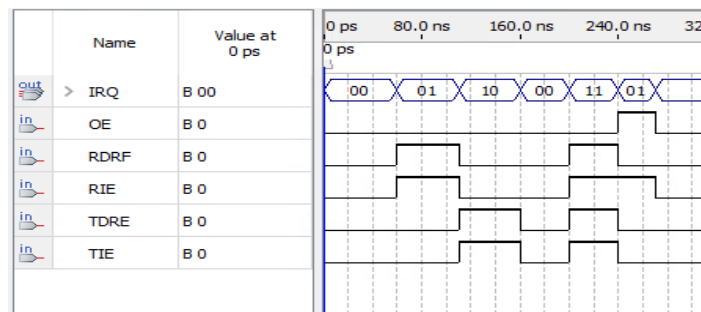


Figure 18: Interrupt Generator

Following the guidance of section 5.5 of the project manual, we were able to successfully simulate the expected behaviour of the interrupt generator. It's clear that when RDRF & RIE are asserted, the receiver bit of the IRQ (i.e the LSB) is set to 1. Similarly, when RDRF & OE are asserted, the receiver bit is also set.

As for the transmitter bit (i.e the MSB), it's set when TDRE & TIE are asserted as per manual instructions.

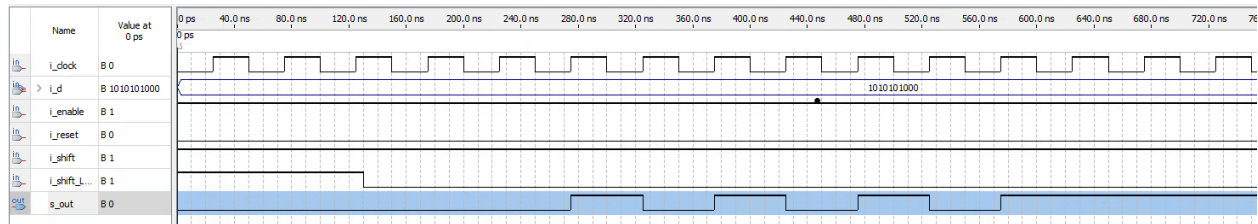


Figure 19: Parallel In Serial Out (PISO)

Figure 19 shows the PISO simulation, a crucial component for the transmitter. As discussed previously, the transmitter must output each debug message serially as per the requirements of the UART. It can be seen that input “i_d” for input data, is set to “1010101000” and s_out shows the output being i_d but in a reverse order. Note that the transmission actually begins after i_shift_load returns to 0.

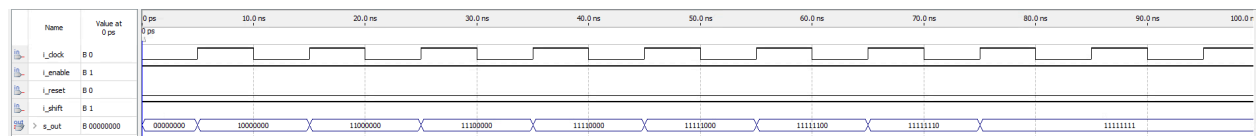


Figure 20: Serial In Parallel Out (SIPO)

Similarly, figure 20 shows the simulation of the SIPO which is a major component used in the receiver. Starts at “00000000” and shifts till “11111111” and returns back to 0.

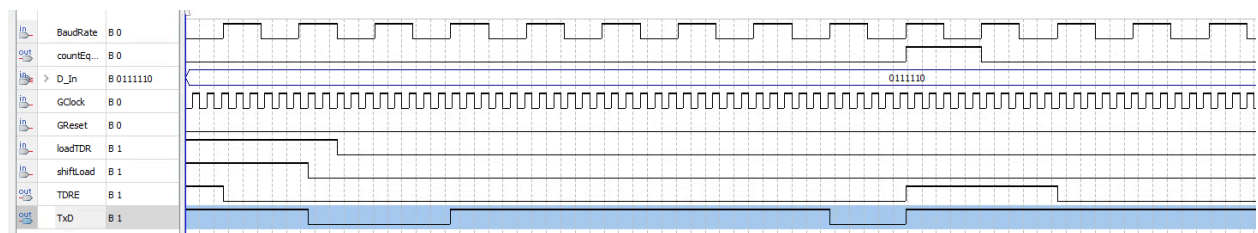


Figure 21: UART Transmitter showcasing parity generation

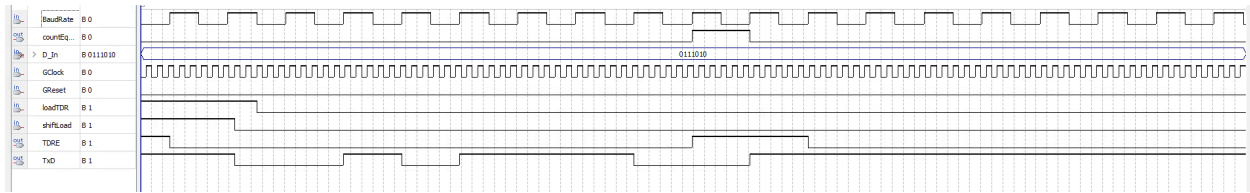


Figure 22: UART Transmitter showcasing parity generation

Figures 21-22 showcase the UART transmitter component including start/stop bits as well as odd parity generation. The transmission starts after shiftLoad returns to 0. The next bit is the start bit “0”, it’s then clear that “0101110” is being transmitted which is the reverse order of D_in. After the 7 bits have been transmitted, we see the parity bit which is then followed by the stop bit “1”. The parity bit is set to 1 when there's an odd number of ones present in D_in (figure 21) and 0 vice versa (figure 22).

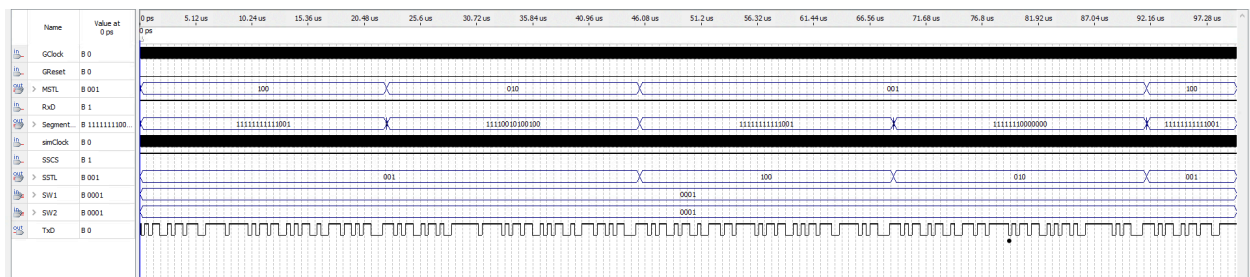


Figure 23: Top level simulation showing all states

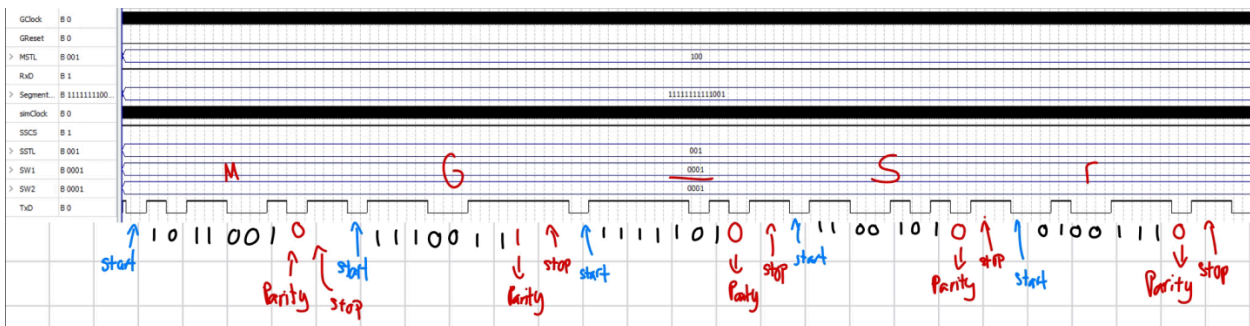


Figure 24: Annotated zoomed in image of SI=100001 (i.e MSTL = 100 & SSTL = 001)

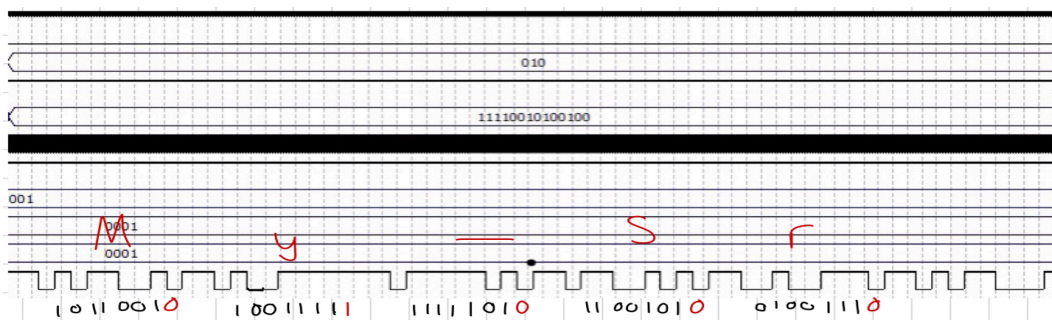


Figure 25: Annotated zoomed in image of SI=100010 (i.e MSTL = 100 & SSTL = 010)

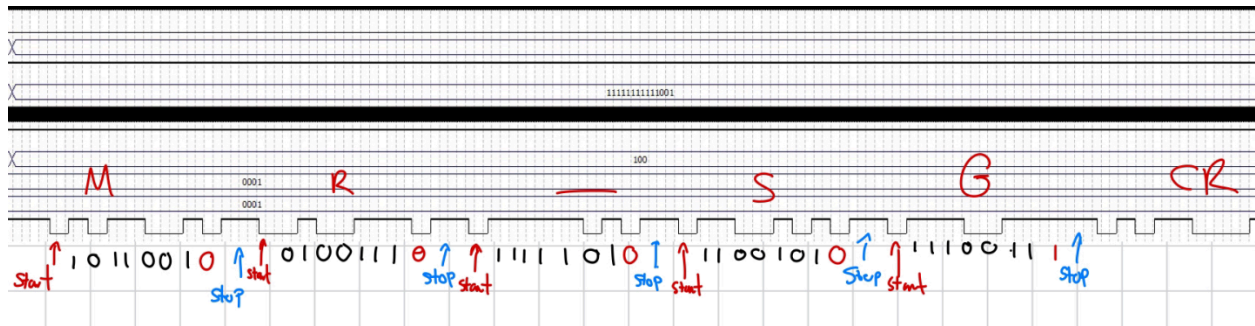


Figure 26: Annotated zoomed in image of SI=001100 (i.e MSTL = 001 & SSTL = 100)

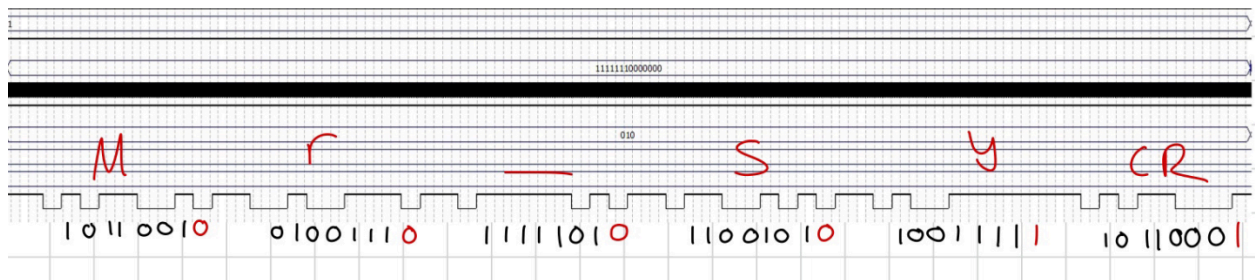


Figure 27: Annotated zoomed in image of SI=001010 (i.e MSTL = 001 & SSTL = 010)

Section 4: Discussion

During the development of the FPGA-based traffic light control system, we encountered several challenges. Initially, our board demo failed to work as expected, particularly with the counter for MSC and SSC in the UART system, which impacted the switching between main and side street traffic light states. Additionally, although the simulations worked fine, once the code was implemented on the Altera board, we encountered issues with incorrect data transmission. While the character "M" was transmitted correctly, subsequent characters were not, which was traced back to logic errors in the VHDL code.

Despite these setbacks, we were able to solve the issues by carefully debugging and refining the UART logic. By using Terra Term to output debug messages, adjusting the baud rate, and ensuring the UART system was properly configured, we were able to resolve the problems. The debug output allowed us to confirm that the FPGA was correctly sending and receiving data, ensuring proper communication between the FPGA and microcontroller. Ultimately, this led to a functional system where both data transmission and traffic light control worked as intended.

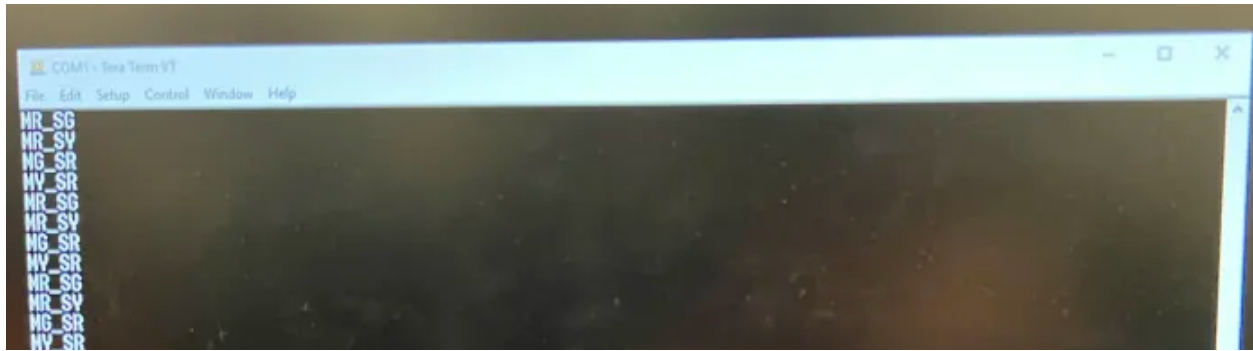


Figure 28: Terra Term showcasing debug messages

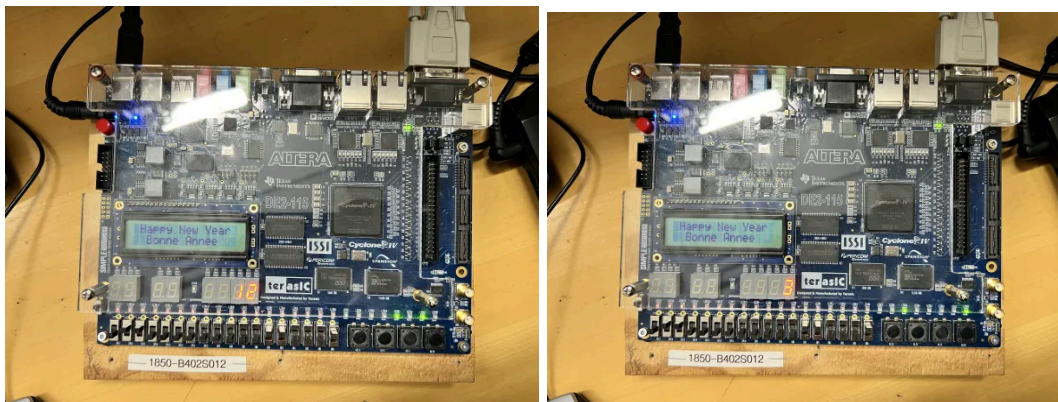


Figure 29: Board Demo

Section 5: Overview

This project aimed to develop a working FPGA-based system to interface with a microcontroller using VHDL and a range of supporting hardware components. The primary goal was to achieve an efficient demonstration of hardware interfacing and data communication between the FPGA and the microcontroller. The system's architecture was designed with various components, including input/output controllers, data buffers, and communication interfaces, to provide a flexible framework for future integration. Throughout the course of the project, multiple approaches were tested to ensure reliable data transmission and debugging capabilities, ensuring the system could be expanded or adapted for other uses. Despite initial challenges with the board demo not functioning as expected, we ultimately achieved success with debug outputs being visible via Terra Term, confirming data transmission and system performance.

Appendix

Code copied directly from VS-Code for clarity.

1. UART Transmitter

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY UARTTransmitter IS
    PORT (
        loadTDR, GReset, GClock, BaudRate, shiftLoad : IN STD_LOGIC;
        D_In : IN STD_LOGIC_VECTOR(6 downto 0);
        TxD, TDRE, countEqualTenOut : OUT STD_LOGIC
    );
END UARTTransmitter;

ARCHITECTURE struct of UARTTransmitter IS

    COMPONENT dFF_8bit
        PORT (
            i_d      : IN  STD_LOGIC_VECTOR(7 downto 0);
            i_reset   : IN  STD_LOGIC;
            i_clock    : IN  STD_LOGIC;
            o_qBar     : OUT STD_LOGIC_VECTOR(7 downto 0)
        );
    END COMPONENT;

    COMPONENT ParallelInSerialOut_8Bit
        PORT (
            i_d      : IN  STD_LOGIC_VECTOR(9 downto 0);
            i_reset   : IN  STD_LOGIC;
            i_enable   : IN  STD_LOGIC;
            i_shift    : IN  STD_LOGIC;
            i_shift_Load : IN  STD_LOGIC;
            i_clock    : IN  STD_LOGIC;
            s_out      : OUT STD_LOGIC
        );
    END COMPONENT;

    COMPONENT parityGen
        PORT (
            i_d : IN  STD_LOGIC_VECTOR(6 downto 0);
            p_out : OUT STD_LOGIC
        );
    END COMPONENT;

    COMPONENT mux_2to1_1bit
        PORT (
            d0 : IN STD_LOGIC;
            d1 : IN STD_LOGIC;
            sel : IN STD_LOGIC;

```



```

        d_out : OUT STD_LOGIC
    );
END COMPONENT;

COMPONENT counter_4bit
PORT (
    CLK      : in STD_LOGIC;
    i_enable  : in STD_LOGIC;
    i_hold    : IN STD_LOGIC;
    i_reset   : IN STD_LOGIC;
    COUNT : out STD_LOGIC_VECTOR (3 downto 0)
);
END COMPONENT;

COMPONENT equality_Comparator_4bit
PORT (
    A, B : IN STD_LOGIC_VECTOR(3 downto 0);
    isEqual : OUT STD_LOGIC
);
END COMPONENT;

signal uartDataBuffer : STD_LOGIC_VECTOR(9 downto 0);
signal latchedData, latchedDataOut : STD_LOGIC_VECTOR(7 downto 0);
signal serialOutputSignal, enableCounter, countReachedTen : STD_LOGIC;
signal countValue : STD_LOGIC_VECTOR(3 downto 0); begin

    uartDataBuffer(9) <= '1';
    uartDataBuffer(0) <= '0';
    latchedData(6 downto 0) <= D_In;
    latchedData(7) <= '0';

    dFF_8bit_inst : dFF_8bit
    PORT MAP (
        i_d => latchedData,
        i_en => loadTDR,
        i_reset => GReset,
        i_clock => GClock,
        o_q => latchedDataOut,
        o_qBar => open
    );

    uartDataBuffer(7 downto 1) <= latchedDataOut(6 downto 0);

    parityGen_inst : parityGen
    PORT MAP (
        i_d => uartDataBuffer(7 downto 1),
        p_out => uartDataBuffer(8)
    ParallelInSerialOut_8Bit_inst : ParallelInSerialOut_8Bit
    PORT MAP (
        i_d => uartDataBuffer,
        i_reset => GReset,
        i_enable => '1',
        i_shift => '1',
        i_shift_Load => shiftLoad,
        i_clock => BaudRate,
        s_out => serialOutputSignal
    );

```

```

mux_2to1_1bit_inst : mux_2to1_1bit
  PORT MAP (
    d0 => serialOutputSignal,
    d1 => '1',
    sel => shiftLoad,
    d_out => TxD
  );

enableCounter <= not countReachedTen;

counter_4bit_inst : counter_4bit
  PORT MAP (
    CLK => BaudRate,
    i_enable => enableCounter,
    i_hold => '0',
    i_reset => GReset,
    COUNT => countValue
  );

equality_Comparator_4bit_inst : equality_Comparator_4bit
  PORT MAP (
    A => countValue,
    B => "1010",
    isEqual => countReachedTen
  );

countEqualTenOut <= countReachedTen;

TDRE <= countReachedTen OR
  (NOT (countValue(3) XOR '0') AND
   NOT (countValue(2) XOR '0') AND
   NOT (countValue(1) XOR '0') AND
   NOT (countValue(0) XOR '0'));

end struct;

```

2. UART

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY UART IS
  PORT (
    RxD, Load, GReset, GClock : IN STD_LOGIC;
    State_Information : IN STD_LOGIC_VECTOR(5 downto 0);
    TxD, CanProceedState : OUT STD_LOGIC
  );
END UART;

ARCHITECTURE struct of UART IS

  COMPONENT UARTTransmitter
    PORT (
      loadTDR, GReset, GClock, BaudRate, shiftLoad : IN STD_LOGIC;
      D_In : IN STD_LOGIC_VECTOR(6 downto 0);
      TxD, TDRE, countEqualTenOut : OUT STD_LOGIC
    );
  END COMPONENT;

```

```

COMPONENT counter_4bit
  PORT (
    CLK      : in STD_LOGIC;
    i_enable  : in STD_LOGIC;
    i_hold   : in STD_LOGIC;
    i_reset   : in STD_LOGIC;
    COUNT : out STD_LOGIC_VECTOR (3 downto 0)
  );
END COMPONENT;

COMPONENT equality_Comparator_4bit
  PORT (
    A, B : in STD_LOGIC_VECTOR(3 downto 0);
    isEqual : out STD_LOGIC
  );
END COMPONENT;

COMPONENT UARTReciever
  PORT (
    RxD, GReset, BClkx8, GClock : in STD_LOGIC;
    RecieveOut : out STD_LOGIC_VECTOR(7 downto 0);
    RDRF : out STD_LOGIC
  );
END COMPONENT;

COMPONENT dFF_8bit
  PORT (
    i_d      : in STD_LOGIC_VECTOR(7 downto 0);
    i_en     : in STD_LOGIC;
    i_reset  : in STD_LOGIC;
    i_clock  : in STD_LOGIC;
    o_q      : out STD_LOGIC_VECTOR(7 downto 0);
    o_qBar   : out STD_LOGIC_VECTOR(7 downto 0)
  );
END COMPONENT;

COMPONENT BaudRateGen
  PORT (
    GClock, GReset : in STD_LOGIC;
    SEL : in STD_LOGIC_VECTOR(2 downto 0);
    BClk, BClkx8 : out STD_LOGIC
  );
END COMPONENT;

COMPONENT InterruptGenerator
  PORT (
    TIE, RIE, RDRF, OE, TDRE: in std_logic;
    IRQ : out std_logic_vector(1 downto 0)
  );
END COMPONENT;

COMPONENT UARTaddressDecode
  PORT (
    Address : in STD_LOGIC_VECTOR(1 downto 0);
    RWbar   : in STD_LOGIC;
    DecodeOut : out STD_LOGIC_VECTOR(5 downto 0)
  );
END COMPONENT;

COMPONENT d_FF

```

```

        PORT (
            i_d, i_en, i_reset      : IN  STD_LOGIC;
            i_clock      : IN  STD_LOGIC;
            o_q          : OUT STD_LOGIC;
            o_qBar       : OUT STD_LOGIC
        );
    END COMPONENT;

    COMPONENT mux_8to1_1bit
    PORT (
        d_in : IN STD_LOGIC_VECTOR(7 downto 0);
        sel  : IN STD_LOGIC_VECTOR(2 downto 0);
        d_out : OUT STD_LOGIC
    );
    END COMPONENT;

    signal reg_SCSROutLatched, reg_SCCRLatched, sig_DataBus, reg_RDROut,
           sig_SCCROut, reg_SCSROut, reg_RDRHeld : STD_LOGIC_VECTOR(7 downto 0);

    signal reg_MSTLState, reg_SSTLState : STD_LOGIC_VECTOR(1 downto 0);

    signal sig_SelectBits, sig_ByteSelector : STD_LOGIC_VECTOR(2 downto 0);

    signal cnt_BytesSent : STD_LOGIC_VECTOR(3 downto 0);

    signal sig_TxD, sig_CanProceedLatched, sig_CountEqualTen, sig_IncByteCounter,
           sig_ResetByteCounter, sig_CanProceed, sig_DoneSendingWord,
           sig_BClk, sig_BClkx8, sig_RDRF, sig_TDRE, sig_TDRELatched : STD_LOGIC;

    signal reg_TxByte, reg_TDRByte, byte_CR, byte_M, byte_Y, byte_R,
           byte_Underscore,
           byte_S, byte_G : STD_LOGIC_VECTOR(6 downto 0);

    signal reg_WordToSend, state_Word1, state_Word2, state_Word3, state_Word4 :
           STD_LOGIC_VECTOR(41 downto 0);

begin
    byte_M <= "1001101";
    byte_Y <= "1111001"; -- lowercase y
    byte_R <= "1110010"; -- lowercase r
    byte_Underscore <= "1011111";
    byte_S <= "1010011";
    byte_G <= "1100111"; -- lowercase g
    byte_CR <= "0001101";

    state_Word1 <= byte_M & byte_G & byte_Underscore & byte_S & byte_R &
byte_CR;
    state_Word2 <= byte_M & byte_Y & byte_Underscore & byte_S & byte_R &
byte_CR;
    state_Word3 <= byte_M & byte_R & byte_Underscore & byte_S & byte_G &
byte_CR;
    state_Word4 <= byte_M & byte_R & byte_Underscore & byte_S & byte_Y &
byte_CR;

    sig_IncByteCounter <= sig_TDRELatched AND sig_CountEqualTen;

    dFF_proceedStateLatched : d_FF
    PORT MAP (
        i_d => sig_CanProceed,
        i_en => '1',
        i_reset => GReset,

```

```

        i_clock => GClock,
        o_q => sig_CanProceedLatched,
        o_qBar => open
    );
    CanProceedState <= sig_CanProceedLatched;

    sig_ResetByteCounter <= GReset OR sig_CanProceedLatched;

    counter_4bit_BytesSent : counter_4bit
    PORT MAP (
        CLK => sig_IncByteCounter,
        i_enable => '1',
        i_hold => '0',
        i_reset => sig_ResetByteCounter,
        COUNT => cnt_BytesSent
    );

    equality_Comparator_4bit_sent6Bytes : equality_Comparator_4bit
    PORT MAP (
        A => cnt_BytesSent,
        B => "0110",
        isEqual => sig_CanProceed
    );

    dFF_8bit_SCCR : dFF_8bit
    PORT MAP (
        i_d => sig_SCCROut,
        i_en => '1',
        i_reset => GReset,
        i_clock => GClock,
        o_q => reg_SCCRLatched,
        o_qBar => open
    );

    reg_SCSROut(6) <= sig_RDRF;

    dFF_8bit_SCSR : dFF_8bit
    PORT MAP (
        i_d => reg_SCSROut,
        i_en => '1',
        i_reset => GReset,
        i_clock => GClock,
        o_q => reg_SCSROutLatched,
        o_qBar => open
    );

    sig_SelectBits <= "000";

    BaudRateGen_inst : BaudRateGen
    PORT MAP (
        GClock => GClock,
        GReset => GReset,
        SEL => sig_SelectBits,
        BClk => sig_BClk,
        BClkx8 => sig_BClkx8
    );

    UARTReciever_inst : UARTReciever
    PORT MAP (
        RxD => sig_TxD,

```

```

        GClock => GClock,
        BClkx8 => sig_BClkx8,
        GReset => GReset,
        RecieveOut => reg_RDROut,
        RDRF => sig_RDRF
    );

dFF_8bit_HoldRecieverOut : dFF_8bit
    PORT MAP (
        i_d => reg_RDROut,
        i_en => '1',
        i_reset => GReset,
        i_clock => sig_RDRF,
        o_q => reg_RDRHeld,
        o_qBar => open
    );

UARTTransmitter_inst : UARTTransmitter
    PORT MAP (
        loadTDR => '1',
        GReset => GReset,
        GClock => GClock,
        BaudRate => sig_BClk,
        shiftLoad => sig_TDRELatched,
        D_In => reg_TxByte,
        TxD => sig_TxD,
        TDRE => sig_TDRE,
        countEqualTenOut => sig_CountEqualTen
    );

d_FF_latchTDRE : d_FF
    PORT MAP (
        i_d => sig_TDRE,
        i_en => '1',
        i_reset => GReset,
        i_clock => GClock,
        o_q => sig_TDRELatched,
        o_qBar => open
    );

TxD <= sig_TxD;

end struct;

```

3. TrafficLightController

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

Entity trafficLighttopLevel IS
    PORT (
        GClock, GReset, SSCS, ProceedNextState : IN STD_LOGIC;
        MSC, SSC : IN STD_LOGIC_VECTOR(3 downto 0);
        MSTL, SSTL : OUT STD_LOGIC_VECTOR(2 downto 0);
        SegmentOut : OUT STD_LOGIC_VECTOR(13 downto 0)
    );
END trafficLighttopLevel;

```

```

architecture struct of trafficLighttopLevel IS

    COMPONENT BCD_to_7Segment
        PORT (
            A_in : IN  STD_LOGIC_VECTOR(3 downto 0);
            seg : OUT STD_LOGIC_VECTOR(13 downto 0)
        );
    END COMPONENT;

    COMPONENT equality_Comparator_4bit
        PORT (
            A, B : IN STD_LOGIC_VECTOR(3 downto 0);
            isEqual : OUT STD_LOGIC
        );
    END COMPONENT;

    COMPONENT counter_4bit
        PORT (
            CLK      : in STD_LOGIC;
            i_enable  : in STD_LOGIC;
            i_hold    : IN STD_LOGIC;
            i_reset   : IN STD_LOGIC;
            COUNT : out STD_LOGIC_VECTOR (3 downto 0)
        );
    END COMPONENT;

    COMPONENT mux_2to1_1bit
        PORT (
            d0 : IN STD_LOGIC;
            d1 : IN STD_LOGIC;
            sel : IN STD_LOGIC;
            d_out : OUT STD_LOGIC
        );
    END COMPONENT;

    COMPONENT mux_2to1_2bit
        PORT (
            sel      : IN  STD_LOGIC;
            d_in1    : IN  STD_LOGIC_VECTOR(1 downto 0);
            d_in2    : IN  STD_LOGIC_VECTOR(1 downto 0);
            d_out    : OUT STD_LOGIC_VECTOR(1 downto 0)
        );
    END COMPONENT;

    COMPONENT mux_4to1_1bit
        PORT (
            d_in : IN STD_LOGIC_VECTOR(3 downto 0);
            sel : IN STD_LOGIC_VECTOR(1 downto 0);
            en : IN STD_LOGIC;
            d_out : OUT STD_LOGIC
        );
    END COMPONENT;

    component d_FF
        Port (
            i_d, i_en, i_reset      : IN  STD_LOGIC;
            i_clock                 : IN   STD_LOGIC;
            o_q, o_qBar             : OUT  STD_LOGIC
        );

```

```

end component;

COMPONENT clk_div
  PORT (
    clock_25Mhz      : IN    STD_LOGIC;
    clock_1MHz       : OUT   STD_LOGIC;
    clock_100KHz     : OUT   STD_LOGIC;
    clock_10KHz      : OUT   STD_LOGIC;
    clock_1KHz       : OUT   STD_LOGIC;
    clock_100Hz      : OUT   STD_LOGIC;
    clock_10Hz       : OUT   STD_LOGIC;
    clock_1Hz        : OUT   STD_LOGIC
  );
END COMPONENT;

signal Segment_Buffer : STD_LOGIC_VECTOR(13 downto 0);
signal Main_Timer, Side_Timer : STD_LOGIC_VECTOR(3 downto 0);
signal Main_STL_Counter, Main_TTL_Counter, Side_STL_Counter,
Side_TTL_Counter, Display_Counter1, Display_Counter2 : STD_LOGIC_VECTOR(3
downto 0);
signal Main_STL_Green, Main_STL_Yellow, Main_STL_Red : STD_LOGIC;
signal Side_STL_Green, Side_STL_Yellow : STD_LOGIC;
signal Main_STL_State, Side_STL_State, Main_STL_State_Next,
Side_STL_State_Next : STD_LOGIC_VECTOR(1 downto 0);
signal Halt_State, Main_STL_Hold, Main_STL_End, Main_TTL_Hold,
Side_STL_Hold, Side_TTL_Hold : STD_LOGIC;
signal Side_STL_End : STD_LOGIC;
signal Side_TTL_End : STD_LOGIC;
signal Main_Timer_End, Side_Timer_End : STD_LOGIC;
begin
  Main_Timer <= "1100";
  Side_Timer <= "1000";

  Main_STL_State <= Main_STL_State_Next;
  Side_STL_State <= Side_STL_State_Next;
  Main_STL_Hold <= (NOT (ProceedNextState AND SCS)) AND Main_STL_End;
  Main_TTL_Hold <= NOT ProceedNextState AND Main_Timer_End;
  Side_STL_Hold <= NOT ProceedNextState AND Side_STL_End;
  Side_TTL_Hold <= NOT ProceedNextState AND Side_TTL_End;

  counter_4bit_Main_STL : counter_4bit
    PORT MAP (
      CLK => GClock,
      i_enable => Main_STL_State(1),
      i_hold => Main_STL_Hold,
      i_reset => GReset,
      COUNT => Main_STL_Counter
    );

  equality_Comparator_4bit_Main_STL : equality_Comparator_4bit
    PORT MAP (
      A => Main_STL_Counter,
      B => MSC,
      isEqual => Main_STL_End
    );

  counter_4bit_Main_TTL : counter_4bit
    PORT MAP (
      CLK => GClock,
      i_enable => Main_STL_State(0),

```



```

        i_hold => Main_TTL_Hold,
        i_reset => GReset,
        COUNT => Main_TTL_Counter
    );

equality_Comparator_4bit_Main_TTL : equality_Comparator_4bit
    PORT MAP (
        A => Main_TTL_Counter,
        B => Main_Timer,
        isEqual => Main_Timer_End
    );

counter_4bit_Side_STL : counter_4bit
    PORT MAP (
        CLK => GClock,
        i_enable => Side_STL_State(1),
        i_hold => Side_STL_Hold,
        i_reset => GReset,
        COUNT => Side_STL_Counter
    );

equality_Comparator_4bit_Side_STL : equality_Comparator_4bit
    PORT MAP (
        A => Side_STL_Counter,
        B => SSC,
        isEqual => Side_STL_End
    );

counter_4bit_Side_TTL : counter_4bit
    PORT MAP (
        CLK => GClock,
        i_enable => Side_STL_State(0),
        i_hold => Side_TTL_Hold,
        i_reset => GReset,
        COUNT => Side_TTL_Counter
    );

equality_Comparator_4bit_Side_TTL : equality_Comparator_4bit
    PORT MAP (
        A => Side_TTL_Counter,
        B => Side_Timer,
        isEqual => Side_TTL_End
    );

Main_STL_Green <= (((NOT SSCS OR NOT Main_STL_End) AND (Main_STL_State(1)
AND NOT Main_STL_State(0))) OR
    (Side_TTL_End OR (NOT Main_STL_State(1) OR NOT Main_STL_State(0))))
    AND NOT (Main_STL_Yellow OR Side_STL_State(1) OR Side_STL_State(0)) AND NOT
Side_STL_Green;

Main_STL_Yellow <= (((SSCS AND Main_STL_End) AND (Main_STL_State(1) AND NOT
Main_STL_State(0))) OR
    (NOT Main_Timer_End AND (NOT Main_STL_State(1) AND Main_STL_State(0))));

Main_STL_Red <= (Main_Timer_End AND (NOT Main_STL_State(1) AND
Main_STL_State(0))) OR
    ((NOT Side_TTL_End OR NOT Side_STL_End) AND (NOT Main_STL_State(1) AND NOT
Main_STL_State(0)));

```

```

Halt_State <= GClock AND (ProceedNextState OR ((NOT Side_STL_State_Next(1)
AND NOT Side_STL_State_Next(0))
AND (NOT Main_STL_State_Next(1) AND NOT Main_STL_State_Next(0))));
d_FF_Main_STL_High : d_FF
PORT MAP (
    i_d => Main_STL_Green,
    i_en => '1',
    i_reset => GReset,
    i_clock => Halt_State,
    o_q => Main_STL_State_Next(1),
    o_qBar => open
);

d_FF_Main_STL_Low : d_FF
PORT MAP (
    i_d => Main_STL_Yellow,
    i_en => '1',
    i_reset => GReset,
    i_clock => Halt_State,
    o_q => Main_STL_State_Next(0),
    o_qBar => open
);

Side_STL_Green <= (Main_Timer_End AND (NOT Side_STL_State(1) AND NOT
Side_STL_State(0))) OR
(NOT Side_STL_End AND (Side_STL_State(1) AND NOT Side_STL_State(0)));

Side_STL_Yellow <= (Side_STL_End AND (Side_STL_State(1) AND NOT
Side_STL_State(0))) OR
(NOT Side_Timer_End AND (NOT Side_STL_State(1) AND Side_STL_State(0)));

SegmentOut <= Segment_Buffer;
end architecture struct;

```

4. UART Receiver

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY UARTReciever IS
    PORT (
        RxD, GReset, BClkx8, GClock : IN STD_LOGIC;
        RecieveOut : OUT STD_LOGIC_VECTOR(7 downto 0);
        RDRF : OUT STD_LOGIC
    );
END UARTReciever;

ARCHITECTURE struct OF UARTReciever IS

    COMPONENT counter_4bit
        PORT (
            CLK : IN STD_LOGIC;
            i_enable : IN STD_LOGIC;
            i_hold : IN STD_LOGIC;
            i_reset : IN STD_LOGIC;
            COUNT : OUT STD_LOGIC_VECTOR (3 DOWNT0 0)

```

```

    );
END COMPONENT;

COMPONENT equality_Comparator_4bit
  PORT (
    A, B : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
    isEqual : OUT STD_LOGIC
  );
END COMPONENT;

COMPONENT SerialInParallelOut_8Bit
  PORT (
    i_reset : IN STD_LOGIC;
    i_enable : IN STD_LOGIC;
    i_shift : IN STD_LOGIC;
    i_clock : IN STD_LOGIC;
    s_out : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
  );
END COMPONENT;

COMPONENT dFF_8bit
  PORT (
    i_d : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
    i_en : IN STD_LOGIC;
    i_reset : IN STD_LOGIC;
    i_clock : IN STD_LOGIC;
    o_q : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
    o_qBar : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
  );
END COMPONENT;

COMPONENT d_FF
  PORT (
    i_d, i_en, i_reset : IN STD_LOGIC;
    i_clock : IN STD_LOGIC;
    o_q : OUT STD_LOGIC;
    o_qBar : OUT STD_LOGIC
  );
END COMPONENT;

SIGNAL countFourVal, countEightVal, numBitsCollected : STD_LOGIC_VECTOR(3
DOWNTO 0);
SIGNAL transitoryPRecieve : STD_LOGIC_VECTOR(7 DOWNTO 0);
SIGNAL isInMiddle, isAtEight, startSample, isStartBit, Latch8, Latch4,
captureBit, finishedCapture, finishedCaptureLatch, resetbitCounter : STD_LOGIC;

BEGIN
  counter_4bit_countmiddle : counter_4bit
  PORT MAP (
    CLK => BClkx8,
    i_enable => isInMiddle,
    i_hold => '0',
    i_reset => GReset,
    COUNT => countFourVal
  );
  counter_4bit_counteight : counter_4bit
  PORT MAP (
    CLK => BClkx8,
    i_enable => Latch8,
    i_hold => '0',

```

```

        i_reset => GReset,
        COUNT => countEightVal
    );
    isInMiddle <= NOT isStartBit AND NOT startSample AND NOT (NOT
(countFourVal(3) XOR '0') AND NOT (countFourVal(2) XOR '0') AND NOT
(countFourVal(1) XOR '1') AND NOT (countFourVal(0) XOR '1'));
    isAtEight <= (NOT isInMiddle AND isStartBit) OR (startSample AND NOT (NOT
(countEightVal(3) XOR '0') AND NOT (countEightVal(2) XOR '1') AND NOT
(countEightVal(1) XOR '1') AND NOT (countEightVal(0) XOR '0')));
    d_FF_latch8 : d_FF
    PORT MAP (
        i_d => isAtEight,
        i_en => '1',
        i_reset => GReset,
        i_clock => BClkx8,
        o_q => Latch8,
        o_qBar => OPEN
    );
    d_FF_latch4 : d_FF
    PORT MAP (
        i_d => isInMiddle,
        i_en => '1',
        i_reset => GReset,
        i_clock => BClkx8,
        o_q => Latch4,
        o_qBar => OPEN
    );
    isStartBit <= NOT RxD AND NOT Latch4 AND NOT (numBitsCollected(3) XOR '0')
AND NOT (numBitsCollected(2) XOR '0') AND NOT (numBitsCollected(1) XOR '0') AND
NOT (numBitsCollected(0) XOR '0');
    equality_Comparator_4bit_startsmpl : equality_Comparator_4bit
    PORT MAP (
        A => numBitsCollected,
        B => "0001",
        isEqual => startSample
    );
    equality_Comparator_4bit_finishsmpl : equality_Comparator_4bit
    PORT MAP (
        A => numBitsCollected,
        B => "1001",
        isEqual => finishedCapture
    );
    d_FF_finishedCap : d_FF
    PORT MAP (
        i_d => finishedCapture,
        i_en => '1',
        i_reset => GReset,
        i_clock => BClkx8,
        o_q => finishedCaptureLatch,
        o_qBar => OPEN
    );
    resetbitCounter <= finishedCaptureLatch OR GReset;
    captureBit <= (NOT Latch4 AND Latch8);
    counter_4bit_countRecieved : counter_4bit
    PORT MAP (
        CLK => captureBit,
        i_enable => '1',
        i_hold => '0',
        i_reset => resetbitCounter,
        COUNT => numBitsCollected
    );

```

```

    );
    SerialInParallelOut_8Bitinst : SerialInParallelOut_8Bit
    PORT MAP (
        i_reset => GReset,
        i_enable => '1',
        i_shift => RxD,
        i_clock => Latch8,
        s_out => transitoryPRecieve
    );
    dFF_8bit_RDR : dFF_8bit
    PORT MAP (
        i_d => transitoryPRecieve,
        i_en => '1',
        i_reset => GReset,
        i_clock => BClkx8,
        o_q => RecieveOut,
        o_qBar => OPEN
    );
    RDRF <= finishedCapture;
END struct;

```

Debuggabletrafficlightcontroller

```

LIBRARY IEEE;
USE ieee.std_logic_1164.ALL;

ENTITY debuggableTrafficLightController IS
    PORT (
        GClock, GReset, SSCS, RxD, simClock : IN STD_LOGIC;
        SW1, SW2 : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
        TxD : OUT STD_LOGIC;
        MSTL, SSTL : OUT STD_LOGIC_VECTOR(2 DOWNTO 0);
        SegmentOut : OUT STD_LOGIC_VECTOR(13 DOWNTO 0)
    );
END debuggableTrafficLightController;

ARCHITECTURE struct OF debuggableTrafficLightController IS

    COMPONENT trafficLighttopLevel
    PORT (
        GClock, GReset, SSCS, ProceedNextState : IN STD_LOGIC;
        MSC, SSC : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
        MSTL, SSTL : OUT STD_LOGIC_VECTOR(2 DOWNTO 0);
        SegmentOut : OUT STD_LOGIC_VECTOR(13 DOWNTO 0)
    );
END COMPONENT;

    COMPONENT clk_div
    PORT (
        clock_25Mhz : IN STD_LOGIC;
        clock_1MHz : OUT STD_LOGIC;
        clock_100KHz : OUT STD_LOGIC;
        clock_10KHz : OUT STD_LOGIC;
        clock_1KHz : OUT STD_LOGIC;
        clock_100Hz : OUT STD_LOGIC;
        clock_10Hz : OUT STD_LOGIC;
        clock_1Hz : OUT STD_LOGIC
    );

```

```

END COMPONENT;

COMPONENT debouncer
  PORT (
    i_reset : IN STD_LOGIC;
    i_clock : IN STD_LOGIC;
    i_raw : IN STD_LOGIC;
    o_clean : OUT STD_LOGIC
  );
END COMPONENT;

COMPONENT mux_2to1_1bit
  PORT (
    d0 : IN STD_LOGIC;
    d1 : IN STD_LOGIC;
    sel : IN STD_LOGIC;
    d_out : OUT STD_LOGIC
  );
END COMPONENT;

COMPONENT UART
  PORT (
    RxD, Load, GReset, GClock : IN STD_LOGIC;
    State_Information : IN STD_LOGIC_VECTOR(5 DOWNTO 0);
    TxD, CanProceedState : OUT STD_LOGIC
  );
END COMPONENT;

SIGNAL ProceedNextState, dividedClock, transitoryClock, debouncedSSCS :
STD_LOGIC;
SIGNAL State_Information : STD_LOGIC_VECTOR(5 DOWNTO 0);
SIGNAL transitorySW1, transitorySW2 : STD_LOGIC_VECTOR(3 DOWNTO 0);
SIGNAL transitoryMSTL, transitorySSTL : STD_LOGIC_VECTOR(2 DOWNTO 0);

BEGIN

  debouncer_SCSS : debouncer
    PORT MAP (
      i_reset => GReset,
      i_clock => GClock,
      i_raw => SSCS,
      o_clean => debouncedSSCS
    );

  debouncer_sw10 : debouncer
    PORT MAP (
      i_reset => GReset,
      i_clock => GClock,
      i_raw => SW1(0),
      o_clean => transitorySW1(0)
    );

  debouncer_sw11 : debouncer
    PORT MAP (
      i_reset => GReset,
      i_clock => GClock,
      i_raw => SW1(1),
      o_clean => transitorySW1(1)
    );

```

```

debouncer_sw12 : debouncer
  PORT MAP (
    i_reset => GReset,
    i_clock => GClock,
    i_raw => SW1(2),
    o_clean => transitorySW1(2)
  );

debouncer_sw13 : debouncer
  PORT MAP (
    i_reset => GReset,
    i_clock => GClock,
    i_raw => SW1(3),
    o_clean => transitorySW1(3)
  );

debouncer_sw20 : debouncer
  PORT MAP (
    i_reset => GReset,
    i_clock => GClock,
    i_raw => SW2(0),
    o_clean => transitorySW2(0)
  );

debouncer_sw21 : debouncer
  PORT MAP (
    i_reset => GReset,
    i_clock => GClock,
    i_raw => SW2(1),
    o_clean => transitorySW2(1)
  );

debouncer_sw22 : debouncer
  PORT MAP (
    i_reset => GReset,
    i_clock => GClock,
    i_raw => SW2(2),
    o_clean => transitorySW2(2)
  );

debouncer_sw23 : debouncer
  PORT MAP (
    i_reset => GReset,
    i_clock => GClock,
    i_raw => SW2(3),
    o_clean => transitorySW2(3)
  );

clk_div_inst : clk_div
  PORT MAP (
    clock_25Mhz => GClock,
    clock_1MHz => dividedClock,
    clock_100KHz => OPEN,
    clock_10KHz => OPEN,
    clock_1KHz => OPEN,
    clock_100Hz => OPEN,
    clock_10Hz => OPEN,
    clock_1Hz => OPEN
  );

```

```

mux_2to1_1bit_chooseClock : mux_2to1_1bit
  PORT MAP (
    d0 => dividedClock,
    d1 => GClock,
    sel => simClock,
    d_out => transitoryClock
  );

trafficLighttopLevel_inst : trafficLighttopLevel
  PORT MAP (
    GClock => transitoryClock,
    GReset => GReset,
    SSSCS => debouncedSSCS,
    ProceedNextState => ProceedNextState,
    MSC => transitorySW1,
    SSC => transitorySW2,
    MSTL => transitoryMSTL,
    SSTL => transitorySSTL,
    SegmentOut => SegmentOut
  );

State_Information(5 DOWNTO 3) <= transitoryMSTL;
State_Information(2 DOWNTO 0) <= transitorySSTL;

MSTL <= transitoryMSTL;
SSTL <= transitorySSTL;

UART_inst : UART
  PORT MAP (
    RxD => RxD,
    Load => '1',
    GReset => GReset,
    GClock => transitoryClock,
    State_Information => State_Information,
    TxD => TxD,
    CanProceedState => ProceedNextState
  );
END struct;

```